The Journal of Open Source Software

REMO: Deterministic Scenario Record and Replay with Modification for ADS Debugging in CARLA

Matthew I Leach [1], **Sanjeetha Pennada** [1], **and Donghwan Shin** [1¶]

1 School of Computer Science, University of Sheffield, United Kingdom ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Open Journals

Reviewers:

- @openjournals

Submitted: 01 January 1970
Published: unpublished

Summary

In simulation-based testing of autonomous driving systems (ADS), the ability to reproducibly recreate and modify driving scenarios is crucial for testing and debugging system performance. For example, when an ADS fails to navigate a complex urban driving scenario, researchers and engineers need to isolate the contributing factors by systematically modifying scenario elements, such as weather conditions, time of day, street lights, traffic signs, or the presence of other vehicles and pedestrians. However, existing simulation platforms like CARLA (Dosovitskiy et al., 2017) often lack the capability to deterministically replay scenarios with modified parameters while maintaining consistent non-player character (NPC) behaviour.

To address this limitation, we present `REMO`, a tool that enables deterministic replay and modification of simulation scenarios within CARLA. `REMO` provides the following key features:

- **Deterministic Scenario Replay**: `REMO` allows users to record and replay any driving scenarios in CARLA while ensuring that all static and dynamic elements, including NPC behaviours, remain consistent across replays, unless intentionally modified.
- **Scenario Modification**: Users can modify various scenario elements/parameters, including environmental conditions (e.g. weather, time of day) and the presence of specific actors (e.g. vehicles, pedestrians), without disrupting the deterministic nature of the replay.
- **ADS-Agnostic Testing**: `REMO` supports scenario replay with or without an ego vehicle (i.e. the autonomous vehicle under test), enabling researchers to test different ADS models under identical NPC conditions for fair comparisons. By enabling deterministic replay and flexible scenario modification, `REMO` facilitates systematic debugging and evaluation of autonomous driving systems.

Statement of Need

Debugging failures in machine learning–enabled autonomous driving systems (ADS) is a complex task that requires the ability to reproducibly recreate and modify driving scenarios (Shin & Pennada, 2024). For example, Arcaini et al. (2022) proposed iterative removal of NPC vehicles to isolate the minimal set of NPCs responsible for triggering a failure, showing that simplified scenarios significantly aid engineers during debugging. However, they focused on NPC removal and did not address the broader need for deterministic scenario replay and modification of other scenario elements (e.g. weather, time of day, buildings). Furthermore, due to the inherent non-determinism of simulation platforms (Chance et al., 2022; Osikowicz et al., 2025), simply re-running a scenario without explicitly recording and replaying it can lead to different outcomes, making it difficult to isolate failure causes. Although CARLA supports record and replay functionality, it lacks the ability to modify scenarios while maintaining deterministic NPC behaviour.

`REMO` is designed to fill this gap by providing a tool that enables deterministic replay of recorded scenarios with the ability to modify various scenario parameters without affecting NPC behaviour. It allows users to systematically explore how different factors contribute

to ADS failures by enabling controlled modifications of the simulation environment. It also supports ADS-agnostic testing by allowing scenario replay without an ego vehicle, facilitating fair comparisons between different ADS models under identical conditions.

State of the Field

Simulation-based evaluation is central to autonomous driving research, with CARLA being the most widely used open-source platform. However, its built-in record-replay functionality cannot replay scenarios without an ego vehicle deterministically and lacks the ability to edit or modify recorded scenarios (Dosovitskiy et al., 2017). Consequently, researchers cannot reliably reproduce autonomous driving scenarios to test the performance of multiple ADSs under identical or variable conditions, which also hinders debugging. This creates a gap in CARLA's functionality: autonomous driving scenarios can be generated but cannot be replayed with modifications.

`REMO` addresses this gap by providing a workflow that combines deterministic, scenario-level record and replay functionality with integrated scenario editing capabilities. It ensures deterministic scenario execution by capturing complete actor trajectories, temporal sequencing, and environmental states. `REMO` decouples scenarios from any specific ADS, allowing replay of the recorded scenario without an ego vehicle, and later injects an ego vehicle driven by a user-specified ADS to evaluate its performance under configurable settings. These settings include, but are not limited to, addition or removal of actors and scenario entities, as well as adjustment of weather and time conditions. By combining these capabilities, `REMO` enables reproducible, repeatable, and modifiable scenario generation that can be evaluated across multiple ADSs.

Software Design

To ensure fair, reproducible, and flexible testing of ADS, we adopted the following design and architectural choices:

- **Separation of scenario and ego vehicle/ADS**: When the ego vehicle is tightly coupled to the scenario, it is difficult to test multiple ADS implementations fairly or to modify the environment without affecting the behaviour of the ego vehicle. Instead, we separate the scenario from the ego vehicle, allowing the same scenario to be reused with different ADS or under modified environmental conditions. This ensures fair comparisons, controlled testing, and flexible scenario modifications without altering the core dynamics.
- **Deterministic scenario replay**: Reproducibility and debugging are essential in ADS testing. While non-deterministic replay could capture real-world variability, it complicates debugging and comparative evaluation. Therefore, we use deterministic replay, which ensures that the same scenario produces identical outcomes across multiple runs. This deterministic design also allows configurable modifications to test system robustness while maintaining consistent and reliable results.
- **Comparison of multiple ADSs**: To test different ADS implementations under identical conditions, we separate the ego vehicle from the scenario. The ego vehicle can be replaced dynamically, enabling controlled comparisons with or without modifying scenario entities or other environmental factors. This design choice supports benchmarking and performance evaluation across multiple ADSs.
- **Control of individual entities**: Each dynamic or structural entity in the scenario is encoded as an independent vector dimension, allowing precise control over its behaviour. This design allows recording, replaying, and modifying each entity independently, which is essential for debugging complex interactions and testing specific failure cases.
- **Architectural design for reproducibility, repeatability, and flexibility**: The design supports recording, deterministic replay, scenario modification, and dynamic injection of ego

vehicles/ADS. This architectural design simplifies recording the scenario, replaying the same scenario multiple times using the user-specified ADS, and modifying the scenarios. This reduces manual effort, accelerates experimentation, and makes it easier for users to reproduce scenarios, analyse failures, and test and evaluate new ADSs.

The package is API-driven, allowing its features to be used independently or in combination. Together, all these design choices prioritise reproducibility, flexibility, and usability, enabling controlled, realistic, and repeatable evaluation of ADS while supporting research, debugging, and development workflows.

Research Impact Statement

REMO addresses a fundamental limitation in ADS: the lack of deterministic scenario replay and controlled scenario modification in CARLA-based testing workflows. It fills a critical gap between high-fidelity simulation and reproducible ADS evaluation by enabling the recording, deterministic replay, and modification of critical driving scenarios.

The deterministic scenario replay, including NPC trajectories, with controlled changes to environmental and structural entities in REMO, improves the reliability and fairness of ADS benchmarking and debugging of failure scenarios. This capability enables researchers and developers to isolate the root causes of ADS failures, compare multiple ADS under identical conditions, and examine the effects of environmental variation on scenario simplification.

Overview of the Tool

Figure 1 illustrates the workflow of the tool in three main phases: scenario setup, scenario recording, and replay with modification.

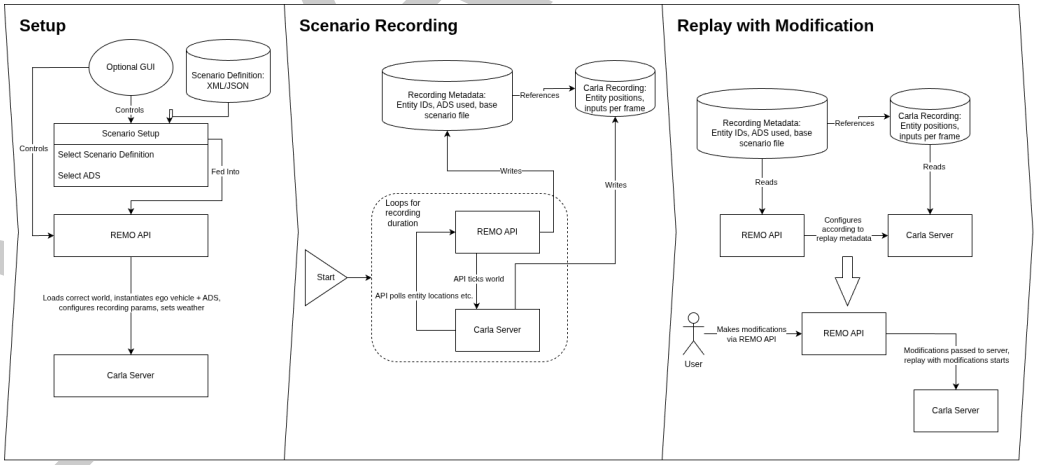

Figure 1: Overall Architecture of REMO

- **Setup**: In the setup phase, users define a scenario using the configuration files, such as XML or JSON, optionally assisted by a graphical user interface. This scenario definition specifies the simulation environment, scenario entities, and the system under test. The REMO API uses this configuration to initialise CARLA by loading the simulation world, spawning the ego vehicle and other entities, configuring the ADS, and applying the specified environmental conditions.
- **Scenario Recording**: During the scenario recording phase, the REMO API controls the execution of the simulation and collects relevant state information from CARLA at each simulation step. The runtime information for all the entities is captured and stored along with static metadata describing the scenario configuration and the ADS under test.

These recordings provide a complete description of the executed scenario, which can be reused for deterministic replay with or without modifications.

- **Replay with Modification**: In the replay with modification phase, previously recorded scenarios can be loaded using stored metadata and replayed deterministically with user-defined modifications. These modifications may include, but are not limited to, changing weather or time conditions, removing NPCs, toggling structural entities, modifying the system under test, or altering the position of NPCs.

Repository and Installation

The tool requires Python 3.8 and CARLA version 0.9.15. The source code of `REMO`, including detailed installation instructions and example scripts, is hosted on GitHub: https://github.com/RSE-Sheffield/REMO.

AI Usage Disclosure

No generative artificial intelligence tools were used in the creation of the software, documentation, or the writing of this paper.

Acknowledgements

This work was supported by the Institute of Information & Communications Technology Planning & Evaluation(IITP) grant funded by the Korea government(MSIT) (No. RS-2025-02218761, 50%) and by the Engineering and Physical Sciences Research Council (EPSRC) [EP/Y014219/1].

References

Arcaini, P., Zhang, X.-Y., & Ishikawa, F. (2022). Less is more: Simplification of test scenarios for autonomous driving system testing. *2022 IEEE Conference on Software Testing, Verification and Validation (ICST)*, 279–290. https://doi.org/10.1109/ICST53961.2022.00037

Chance, G., Ghobrial, A., McAreavey, K., Lemaignan, S., Pipe, T., & Eder, K. (2022). On determinism of game engines used for simulation-based autonomous vehicle verification. *IEEE Transactions on Intelligent Transportation Systems*, *23*(11), 20538–20552. https://doi.org/10.1109/TITS.2022.3177887

Dosovitskiy, A., Ros, G., Codevilla, F., Lopez, A., & Koltun, V. (2017). CARLA: An open urban driving simulator. In S. Levine, V. Vanhoucke, & K. Goldberg (Eds.), *Proceedings of the 1st annual conference on robot learning* (Vol. 78, pp. 1–16). PMLR. https://proceedings.mlr.press/v78/dosovitskiy17a.html

Osikowicz, O., McMinn, P., & Shin, D. (2025). Empirically evaluating flaky tests for autonomous driving systems in simulated environments. *2025 IEEE/ACM International Flaky Tests Workshop (FTW)*, 13–20. https://doi.org/10.1109/FTW66604.2025.00009

Shin, D., & Pennada, S. (2024). Towards simplification of failure scenarios for machine learning-enabled autonomous systems. *2024 IEEE 24th International Conference on Software Quality, Reliability, and Security Companion (QRS-c)*, 1089–1090. https://doi.org/10.1109/QRS-C63300.2024.00143